

一、整合概述

libpcap 是一个系统无关的用户级数据包捕获接口，为网络流量捕获提供可移植的框架。它是 tcpdump、Wireshark 等网络工具的基础依赖库。本文档记录了 libpcap 在 OpenHarmony hispark_hi3519dv500_linux 产品中的整合过程。

版本信息：

- libpcap 版本：1.10.4
- OpenHarmony 版本：4.0-Release
- 目标产品：hispark_hi3519dv500_linux (ipcamera_hispark_hi3519dv500_linux)
- 目标架构：ARM64 (aarch64)
- 编译工具链：aarch64-v01c01-linux-musl-gcc
- 内核：Linux 5.10
- 系统类型：small
- 编译日期：2026-05-18

二、信息收集

1. 组件基本信息

项目	值
组件名称	libpcap
版本	Release_1.10.4
许可证	BSD-3-Clause
subsystem	thirdparty
adapted_system_type	small, standard
构建产物	libpcap_shared.so (共享库)
部署位置	/system/lib/libpcap_shared.so

2. 组件依赖

```
libpcap
└─ musl (sysroot_lite) - C 标准库 (lite 系统分支显式依赖)
```

3. 关键源文件清单

文件位置： `third_party/libpcap/`

类型	文件
构建配置	BUILD.gn, bundle.json
平台相关源文件	pcap-linux.c, fad-getad.c
模块源文件	pcap-usb-linux.c, pcap-netfilter-linux.c
通用源文件	pcap.c, gencode.c, optimize.c, nametoaddr.c, etherent.c, fmtutils.c, pcap-util.c, savefile.c, sf-pcap.c, sf-pcapng.c, pcap-common.c, pcap-usb-linux-common.c, bpf_image.c, bpf_filter.c, bpf_dump.c
生成源文件	scanner.c, grammar.c
编译配置	config.h

三、整合步骤

1. 产品配置修改

文件： `vendor/hisilicon/hispark_hi3519dv500_linux/config.json`

在 `thirdparty` 子系统的 `components` 数组中添加 libpcap 组件条目。

修改前：

```
{
  "subsystem": "thirdparty",
  "components": [
    { "component": "busybox", "features":[] },
    { "component": "libmnl", "features":[] },
    { "component": "ethtool", "features":[] }
  ]
}
```

修改后：

```
{
  "subsystem": "thirdparty",
  "components": [
    { "component": "busybox", "features":[] },
    { "component": "libmnl", "features":[] },
    { "component": "ethtool", "features":[] },
    { "component": "libpcap", "features":[] }
  ]
}
```

2. 构建脚本修改

文件: `third_party/libpcap/BUILD.gn`

原始 BUILD.gn 包含两个编译标志 `-D_UAPI_LINUX_SOCKET_H` 和 `-D__kernel_sa_family_t=sa_family_t`, 这两个标志在 hispark_taurus_linux (ARM 32-bit) 平台可正常工作, 但在 hispark_hi3519dv500_linux (ARM64/musl) 平台会导致编译失败。需要在 BUILD.gn 两处移除这两个标志。

2.1 common_config 配置段 (标准系统分支, 第 16-22 行)

修改前:

```
config("common_config") {
  cflags = [
    "-fpic",
    "-D_UAPI_LINUX_SOCKET_H", # 防止 sockaddr_storage 重定义
    "-D__kernel_sa_family_t=sa_family_t" # 定义内核类型
  ]
  defines = [ "BUILDING_PCAP", "pcap_EXPORTS", "HAVE_CONFIG_H", "_GNU_SOURCE" ]
}
```

修改后:

```
config("common_config") {
  cflags = [
    "-fpic"
  ]
  defines = [ "BUILDING_PCAP", "pcap_EXPORTS", "HAVE_CONFIG_H", "_GNU_SOURCE" ]
}
```

2.2 lite_library 配置段 (lite 系统分支, 第 49-60 行)

修改前:

```
  cflags = [
    "-fpic",
    "-fPIC",
    "-D_UAPI_LINUX_SOCKET_H", # 防止 sockaddr_storage 重定义
    "-D__kernel_sa_family_t=sa_family_t", # 定义内核类型
    "-DBUILDING_PCAP",
    "-Dpcap_EXPORTS",
    "-DHAVE_CONFIG_H",
    "-D_GNU_SOURCE"
  ]
```

修改后:

```

cflags = [
    "-fpic",
    "-fPIC",
    "-DBUILDING_PCAP",
    "-Dpcap_EXPORTS",
    "-DHAVE_CONFIG_H",
    "-D_GNU_SOURCE"
]

```

3. 修改后的完整 BUILD.gn

```

# Copyright (C) 2021 Huawei Device Co., Ltd.
# Licensed under the Apache License, Version 2.0 (the "License");

config("common_config") {
    cflags = [
        "-fpic"
    ]
    defines = [ "BUILDING_PCAP", "pcap_EXPORTS", "HAVE_CONFIG_H", "_GNU_SOURCE" ]
}

common_include_dirs = [ "//third_party/libpcap" ]
libpcap_sources = []
PLATFORM_C_SRC = [ "pcap-linux.c", "fad-getad.c" ]
MODULE_C_SRC = [ "pcap-usb-linux.c", "pcap-netfilter-linux.c" ]
REMOTE_C_SRC = []
COMMON_C_SRC = [ "pcap.c", "gencode.c", "optimize.c", "nametoaddr.c", "etherent.c",
    "fmtutils.c", "pcap-util.c", "savefile.c", "sf-pcap.c", "sf-pcapng.c",
    "pcap-common.c", "pcap-usb-linux-common.c", "bpf_image.c", "bpf_filter.c", "bpf_dump.c" ]
GENERATED_C_SRC = [ "scanner.c", "grammar.c" ]

libpcap_sources += PLATFORM_C_SRC
libpcap_sources += MODULE_C_SRC
libpcap_sources += REMOTE_C_SRC
libpcap_sources += COMMON_C_SRC
libpcap_sources += GENERATED_C_SRC

if (defined(ohos_lite))
{
    import("//build/lite/config/component/lite_component.gni")
    lite_library("libpcap_shared") {
        target_type = "shared_library"
        sources = libpcap_sources
        include_dirs = common_include_dirs

        cflags = [
            "-fpic",
            "-fPIC",
            "-DBUILDING_PCAP",
            "-Dpcap_EXPORTS",
            "-DHAVE_CONFIG_H",
            "-D_GNU_SOURCE"

```

```

    ]

    deps = [ "//third_party/musl:sysroot_lite" ]
}
else
{
    import("//build/ohos.gni")
    import("//build/ohos/ndk/ndk.gni")
    ohos_shared_library("libpcap_shared") {
        sources = libpcap_sources
        include_dirs = common_include_dirs
        configs = [ ":common_config" ]
        deps = []
        part_name = "libpcap"
        install_enable = true
        install_images = [ "system" ]
        subsystem_name = "thirdparty"
    }
}
}

```

四、编译验证

编译命令

```

# 单独编译 libpcap（增量编译）
hb build //third_party/libpcap:libpcap_shared

# 或清理后全量编译
rm -rf out && hb build //third_party/libpcap:libpcap_shared

```

编译结果

```

[OHOS INFO] build success
[OHOS INFO] Cost time: 0:01:06

```

生成文件

```

$ find out/ -name "libpcap_shared.so" -exec ls -lh {} \;
-rwxrwxr-x 283K
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/usr/lib/libpcap_shared.so
-rwxrwxr-x 308K
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/unstripped/usr/lib/libpcap_shared.so
-r-xr-xr-x 283K
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/rootfs/usr/lib/libpcap_shared.so

```

ELF 架构验证

```
$ file out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/usr/lib/libpcap_shared.so
libpcap_shared.so: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV),
dynamically linked, stripped
```

符号验证

```
$ nm -D
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/unstripped/usr/lib/libpcap_shared
.so | grep -E
"pcap_open_live|pcap_close|pcap_compile|pcap_loop|pcap_findalldevs|pcap_lib_version"
00000000000103d0 T pcap_findalldevs
000000000000cee8 T pcap_lib_version
0000000000010fb4 T pcap_loop
00000000000126ac T pcap_open_live
0000000000012648 T pcap_close
00000000000137fc T pcap_compile
```

关键导出符号：

符号	地址	说明
pcap_open_live	0x126ac	打开网络设备进行抓包
pcap_close	0x12648	关闭抓包会话
pcap_compile	0x137fc	编译 BPF 过滤规则
pcap_compile_nopcap	0x13ea4	在无设备情况下编译 BPF
pcap_loop	0x10fb4	循环捕获数据包
pcap_findalldevs	0x103d0	查找所有可用网络设备
pcap_lib_version	0x0cee8	获取库版本信息

依赖验证

```
$ readelf -d
out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/usr/lib/libpcap_shared.so | grep
NEEDED
0x0000000000000001 (NEEDED) shared library: [libc.so]
```

仅依赖 musl libc，无其他外部依赖。

编译统计

指标	数值
源文件数	21 个 C 文件
编译时间	约 1 分钟 (增量)
stripped 大小	283 KB
unstripped 大小	308 KB
架构	ARM 64-bit (aarch64)
依赖库	libc.so

五、问题解决记录

问题 1: sockaddr_storage 类型未定义 (编译失败)

错误信息:

```
FAILED: obj/third_party/libpcap/libpcap_shared.etherent.o
ccache aarch64-v01c01-linux-musl-gcc ... -D_UAPI_LINUX_SOCKET_H ... -c
../../../../third_party/libpcap/etherent.c

In file included from sysroot/usr/include/netdb.h:9,
                 from ../../../../third_party/libpcap/pcap/socket.h:59,
                 from ../../../../third_party/libpcap/pcap/pcap.h:130,
                 from ../../../../third_party/libpcap/pcap-int.h:41,
                 from ../../../../third_party/libpcap/etherent.c:32:
sysroot/usr/include/netinet/in.h:281:26: error: field 'gr_group' has incomplete type
281 | struct sockaddr_storage gr_group;
sysroot/usr/include/netinet/in.h:286:26: error: field 'gsr_group' has incomplete type
286 | struct sockaddr_storage gsr_group;
sysroot/usr/include/netinet/in.h:287:26: error: field 'gsr_source' has incomplete type
287 | struct sockaddr_storage gsr_source;
sysroot/usr/include/netinet/in.h:292:26: error: field 'gf_group' has incomplete type
292 | struct sockaddr_storage gf_group;
sysroot/usr/include/netinet/in.h:295:26: error: array type has incomplete element type
'struct sockaddr_storage'
295 | struct sockaddr_storage gf_slist[1];
```

影响的源文件: 所有 21 个 C 源文件 (etherent.c, fmtutils.c, pcap.c 等全部报错)

根因分析:

struct sockaddr_storage 在 sysroot 头文件中有两处定义, 均被条件编译跳过:

定义点 1 — sys/socket.h 第 378-382 行:

```

#ifdef __MUSL__           // musl 环境下 __MUSL__ 已定义 → 此分支跳过
struct sockaddr_storage {
    sa_family_t ss_family;
    char __ss_padding[128-sizeof(long)-sizeof(sa_family_t)];
    unsigned long __ss_align;
};
#endif

```

musl libc 在编译时定义了 `__MUSL__` 宏，导致 `sys/socket.h` 中的 `struct sockaddr_storage` 定义被跳过。

定义点 2 — `linux/socket.h` 第 12 行:

```

#ifndef _UAPI_LINUX_SOCKET_H // BUILD.gn 中 -D_UAPI_LINUX_SOCKET_H 触发 → 整个文件跳过
#define _UAPI_LINUX_SOCKET_H
struct sockaddr_storage {
    __kernel_sa_family_t ss_family;
    char __data[_K_SS_MAXSIZE - sizeof(unsigned short)];
};
#endif

```

BUILD.gn 中的 `-D_UAPI_LINUX_SOCKET_H` 标志使得 `linux/socket.h` 的 include guard 在 include 之前就已满足，整个文件被跳过。

include 链路:

```

etherent.c
→ pcap-int.h
→ pcap/pcap.h
→ pcap/socket.h
→ netdb.h
→ netinet/in.h    ← 此处使用 struct sockaddr_storage，但类型从未被定义

```

虽然 `netinet/in.h` 自身 `#include <sys/socket.h>`，但由于上述两个定义点都被跳过，`struct sockaddr_storage` 实际上从未被定义。

解决方案:

移除 BUILD.gn 中的 `-D_UAPI_LINUX_SOCKET_H` 和 `-D__kernel_sa_family_t=sa_family_t` 编译标志。

移除后 `linux/socket.h` 可以正常被包含，提供 `struct sockaddr_storage` 的定义。同时 `sys/socket.h` 中同名结构体被 `#ifndef __MUSL__` 保护（在 musl 环境下跳过），不会产生重定义冲突。

为什么 hispark_taurus_linux 不需要此修改:

hispark_taurus_linux 使用 ARM 32-bit 工具链，其 musl sysroot 头文件的 `sys/socket.h` 中可能未使用 `#ifndef __MUSL__` 保护 `struct sockaddr_storage` 定义，或者 `__MUSL__` 宏未定义，因此 `sys/socket.h` 自身就能提供该类型定义，不依赖 `linux/socket.h`。

六、与 taurus 平台差异对比

项目	hispark_taurus_linux (原始)	hispark_hi3519dv500_linux (本次)
架构	ARM 32-bit	ARM 64-bit (aarch64)
编译工具链	arm-linux-ohos-gcc	aarch64-v01c01-linux-musl-gcc
系统类型	small	small
libc	musl	musl
BUILD.gn 修改	无需修改	移除 <code>-D_UAPI_LINUX_SOCKET_H</code> 和 <code>-D__kernel_sa_family_t=sa_family_t</code>
编译命令	<code>./build.sh --product-name ipcamera_hispark_taurus_linux --build-target libpcap_shared</code>	<code>hb build //third_party/libpcap:libpcap_shared</code>
产物大小	247 KB	283 KB
ELF 类型	ELF 32-bit LSB shared object, ARM	ELF 64-bit LSB shared object, ARM aarch64

七、使用方法

1. 链接 libpcap

在应用 BUILD.gn 中添加依赖：

```
deps = [ "//third_party/libpcap:libpcap_shared" ]
```

2. 头文件引用

```
#include <pcap/pcap.h>
```

3. 基本使用示例

```
#include <pcap/pcap.h>
#include <stdio.h>

int main() {
    char errbuf[PCAP_ERRBUF_SIZE];

    printf("libpcap version: %s\n", pcap_lib_version());

    pcap_if_t *alldevs;
    if (pcap_findalldevs(&alldevs, errbuf) == -1) {
        fprintf(stderr, "Error: %s\n", errbuf);
        return 1;
    }
    for (pcap_if_t *d = alldevs; d; d = d->next) {
```

```
    printf(" %s\n", d->name);
}
pcap_freealldevs(alldevs);
return 0;
}
```

4. 在设备上部署

```
hdc file send libpcap_shared.so /usr/lib/
hdc shell /data/local/tmp/your_app
```

八、与其他组件的关系

依赖

- **musl libc (sysroot_lite)**: C 标准库, lite 系统分支的显式依赖

上游组件

libpcap 可被以下组件依赖:

1. **tcpdump**: 命令行抓包工具, 直接依赖 libpcap
2. **自定义网络分析工具**: 流量监控、协议分析等

配合使用

```
# 使用 busybox 查看网络接口
busybox ifconfig -a

# 使用 ethtool 查看网卡状态
ethtool eth0

# 使用 tcpdump (依赖 libpcap) 在指定接口抓包
tcpdump -i eth0 -w capture.pcap
```

九、注意事项

1. 重新下载组件后需重新修改 BUILD.gn

libpcap 组件通过 repo/git 管理代码, 重新下载或 `repo sync` 后 BUILD.gn 会被恢复为原始版本 (包含 `-D_UAPI_LINUX_SOCKET_H`), 需要重新应用修改。

2. 头文件兼容性

ARM64/musl 平台编译时需注意 musl libc 与内核头文件的兼容性。 `sys/socket.h` 中 `struct sockaddr_storage` 被 `#ifndef __MUSL__` 保护, 在 musl 环境下不会定义, 需要依赖 `linux/socket.h` 提供定义。

3. 部署位置

编译产物自动安装到 `rootfs/usr/lib/libpcap_shared.so` , 运行时需确保该路径在 `LD_LIBRARY_PATH` 中。

4. 权限要求

使用 libpcap 进行数据包捕获需要 root 权限或 `CAP_NET_RAW` 能力。

5. bundle.json 无需修改

`third_party/libpcap/bundle.json` 中的 `adapted_system_type` 已包含 `"small"` , 子系统映射 `thirdparty` → `third_party/libpcap` 已在 `build/subsystem_config.json` 中配置, 无需额外修改。

十、修改文件清单

文件	修改内容	必要性
<code>vendor/hisilicon/hispark_hi3519dv500_linux/config.json</code>	thirdparty subsystem 添加 libpcap 组件	必须
<code>third_party/libpcap/BUILD.gn</code> (common_config 段)	移除 <code>-D_UAPI_LINUX_SOCKET_H</code> 和 <code>-D__kernel_sa_family_t=sa_family_t</code>	必须
<code>third_party/libpcap/BUILD.gn</code> (lite_library 段)	移除 <code>-D_UAPI_LINUX_SOCKET_H</code> 和 <code>-D__kernel_sa_family_t=sa_family_t</code>	必须

十一、参考资源

- libpcap 官网: <https://www.tcpdump.org/>
- libpcap 源码: <https://github.com/the-tcpdump-group/libpcap>
- OpenHarmony 文档: <https://gitee.com/openharmony>